

Distributed LLM Inference on a Single H100

Blueprint 1 results: cluster setup, capacity, benchmarks and agent validation

Model	Qwen2.5-7B-Instruct (FP8) on vLLM 0.29 with LMCache 0.5.5 and Mooncake 0.3.13
Cluster	2 prefill + 3 decode vLLM pods on one HAMi-sliced NVIDIA H100 PCIe 80 GB (Lambda Cloud)
Configuration	C: kv_router with admission queue and prefill→decode hop, LMCache/Mooncake KV tier. Compared throughput with the same stack without the KV tier (C-nokv)
Agent workload	CrewAI data-analyst crew (3 agents, 9 tools) driving the cluster through a LiteLLM gateway
Measured	26 September 2026
Data	<code>bench-results/</code> : <code>vllm bench serve</code> JSON, per-pod metrics snapshots, <code>summary.csv</code> , <code>agent_runs.csv</code> , <code>agent_questions.csv</code> , per-call agent traces; Grafana dashboard "Blueprint 1 — Inference stack"

Contents

- [Executive summary](#)
- [Setup overview](#)
- [Part 1. Cluster setup](#)
- [Part 2. Capacity on paper](#)
- [Part 3. Benchmarks: rate4 and rateinf](#)
- [Part 4. Validation with agent runs](#)
- [Appendix. Reproducing the runs](#)

Executive summary

- A complete inference stack (gateway, router with admission queue, disaggregated prefill/decode vLLM pods, a shared LMCache/Mooncake KV tier, observability) runs on one H100 split into five GPU slices. It was measured with synthetic benchmarks, with and without the KV tier, and validated with a tool-calling agent workload.
- **The KV tier works, and holds far more reusable KV than the GPUs.** Mooncake exposes a 37 GiB pool ($\approx$ 693k tokens of KV, about 3 $\times$ the decode pods' combined GPU KV cache). Across the three benchmarks it stored 931 objects (13.7 GB), with no failed writes and no evictions, and served 15–33% of prompt tokens back to vLLM from the tier.
 - **Under a burst, the stack with the KV tier is faster.** For 400 simultaneous requests, output throughput rose 9% (2,424 $\rightarrow$ 2,646 tokens/s), mean inter-token latency fell 41%, p95 inter-token latency 80% (197 $\rightarrow$ 40 ms), median end-to-end latency 19%, and preemptions went from 2 to 0. Part of this comes from more even GPU memory allocation in that deployment and from prompt reuse between runs, so the gain is an upper bound on the KV tier's own contribution.
 - **At light load the KV tier costs time to first token.** At 4 requests/s, median TTFT rose from 222 to 254 ms (p95 273 $\rightarrow$ 381 ms). In multi-turn chats, p95 TTFT rose from 311 to 492 ms, while throughput was

unchanged. Every request now writes and looks up KV in the tier, and at this load the saved computation doesn't pay for that.

- **Bursts are absorbed by the router's admission queue, as designed.** It admits up to 64 requests per decode pod and queued ≈ 210 of a 400-request burst, so vLLM itself never queued. No request failed.
- **Prefix caching drives conversational and agent performance.** Multi-turn chats hit the GPU prefix cache 85% of the time, and agent runs 78–93%, because `kv_router` keeps each conversation on the pod that already holds its prefix.
- **The agent workload became reliable step by step: 1/12 $\rightarrow$ 6/12 correct.** Per-call traces showed the 7B model writing tool calls as text, batching tool calls that the parser rejects all-or-nothing, and guessing the schema. Sampling changes, guardrails and one tool call per reply fixed these mechanical failures; the remaining misses are reasoning errors. The agent runs were measured on the stack without the KV tier.

Setup overview

Model and serving engine

Item	Value
Model	Qwen/Qwen2.5-7B-Instruct (7.6 B parameters), served as qwen7b
Weights	FP8 (native on H100), $\approx$ 8.7 GB per replica
KV cache	BF16, 16-token blocks, 56 KiB per token : $2 (K,V) \times 28 \text{ layers} \times 4 \text{ KV heads} \times 128 \text{ dims} \times 2 \text{ bytes}$
Context	max_model_len 16,384 tokens; max_num_seqs 64 per pod
Engine	vLLM 0.29.0 with LMCache 0.5.5 (image lmcache/vllm-openai:v0.5.5, CUDA 13.0), prefix caching on
KV tier	LMCache $\rightarrow$ Mooncake (mooncake-transfer-engine-cuda13 0.3.13.post1), TCP transport, 256-token chunks
Tool calling	--enable-auto-tool-choice --tool-call-parser hermes (Qwen2.5 returns OpenAI tool_calls)
GPU share	gpu_memory_utilization 0.90 of each pod's HAMi slice

Access path

Clients use the OpenAI API against a **LiteLLM gateway** (API-key auth, response cache off, NodePort 30400, reached from the laptop through an SSH tunnel as localhost:4000). The gateway forwards to one **router Service**, behind which kv_router runs the admission queue, prefix-aware routing and the prefill $\rightarrow$ decode hop. Benchmarks call the router directly from inside the cluster; the agent goes through the gateway.

Agent system

A CrewAI 1.15.22 crew runs on the laptop (Python 3.12), answers business questions about a synthetic e-commerce database, and is graded against ground truth.

- **Agents, in sequence:** Data Planner (explores the schema), SQL Analyst (writes, checks and fixes SQLite), Reviewer (verifies independently and ends with FINAL_ANSWER).
- **Nine local tools:** list_tables, describe_table, sample_rows, distinct_values, run_sql (read-only), column_stats (median/p90), calculator (AST, no eval), save_note, read_notes.
- **Data:** a seeded SQLite database (500 customers, 60 products, 3,731 orders, 8,022 order lines, 1,500 support tickets) with deliberate traps: only completed orders count as revenue, discounts apply per order, costs live on another table.
- **Questions:** 12 with known answers (1 easy, 4 medium, 7 hard).
- **Settings:** temperature 0.3, top_p 0.8, top_k 20, repetition penalty 1.05, $\leq$ 1,024 tokens per call, 15 steps per agent, generation stops at </tool_call> (one tool call per reply), guardrails with 2 retries.

Analyst Crew — CrewAI agents on the Blueprint 1 cluster

3 agents · 9 local tools · up to ~45 tool calls per question · every answer graded against ground truth

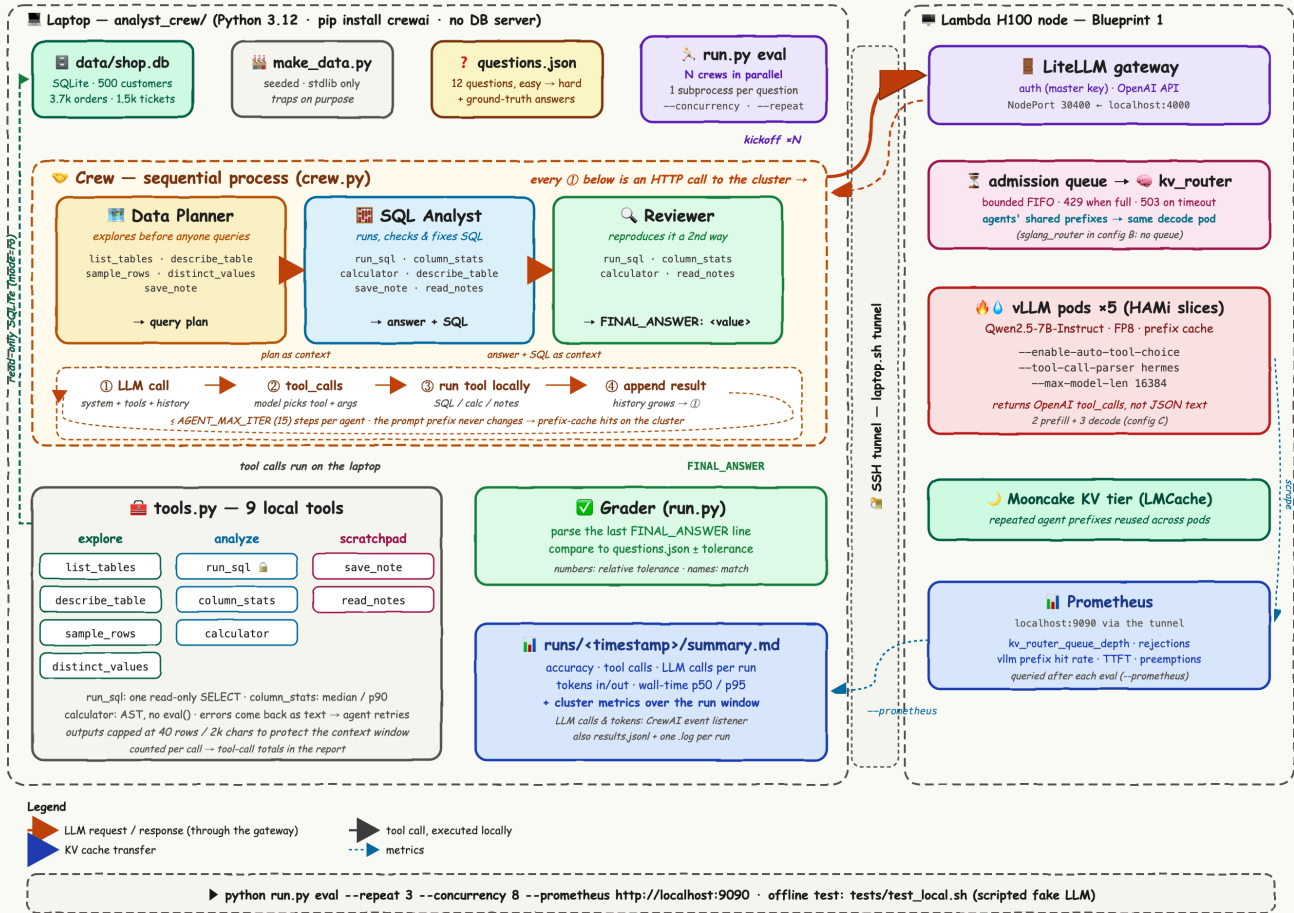


Figure 1. The laptop-side crew, its tools, grader and report, connected through the SSH tunnel to the gateway, router and vLLM pods.

Part 1. Cluster setup

Everything runs on one Lambda Cloud instance with a single **NVIDIA H100 PCIe 80 GB** (driver 580.105.08), managed as a one-node **k3s** Kubernetes cluster. **HAMi** splits the GPU into memory and compute slices so five vLLM pods share it; the NVIDIA GPU Operator provides device monitoring; kube-prometheus-stack and Grafana collect metrics every 15 s.

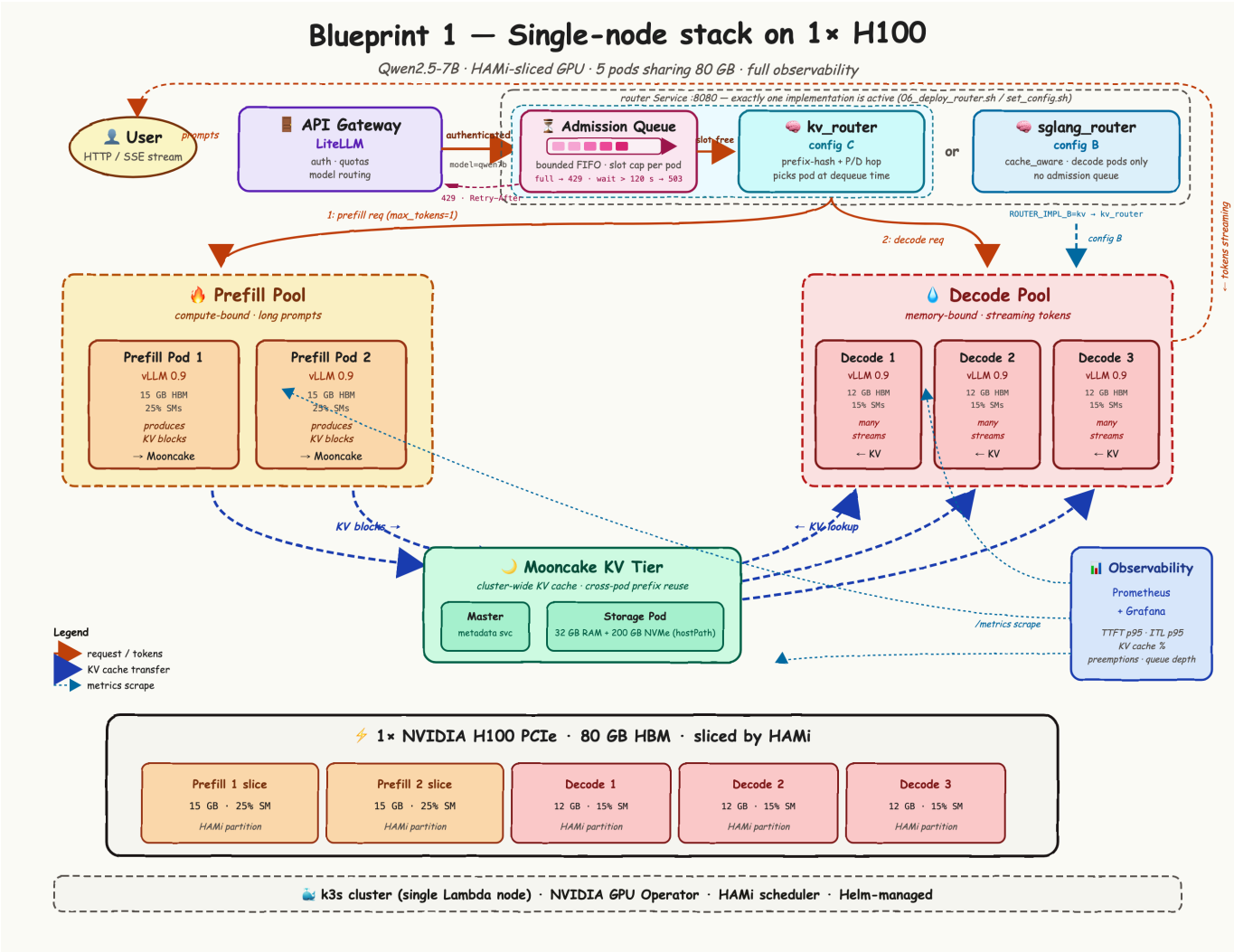


Figure 2. The router Service fronts `kv_router` (configuration C) or `sglang_router` (configuration B). Pod boxes show the original plan's slice sizes and vLLM version; the as-deployed values are in Table 1.

As deployed

Table 1. Inference pods, their GPU slices, and the KV cache vLLM allocated (`vllm:cache_config_info`).

Pod	Role	GPU memory	Compute (SMs)	KV cache (tokens)	16k-token sequences that fit
vllm-prefill-0	prefill	16 GB	25%	105,840	6.5

Pod	Role	GPU memory	Compute (SMs)	KV cache (tokens)	16k-token sequences that fit
vllm-prefill-1	prefill	16 GB	25%	105,840	6.5
vllm-decode-0	decode	14 GB	15%	72,128	4.4
vllm-decode-1	decode	14 GB	15%	72,128	4.4
vllm-decode-2	decode	14 GB	15%	72,128	4.4
Total		74 of 80 GB	95%	428,064	

GPU memory allocation depends on how pods start. vLLM sizes its KV cache from the free memory it measures at start-up. In the C-nokv deployment all five pods started at once, and two received less than their identical peers: `vllm-decode-2` got 39,440 tokens (55% of its peers) and `vllm-prefill-1` 73,136 (69%). In the deployment with the KV tier, a rolling update started the pods one at a time, and every pod got its full share. Staggering start-up (or fixing the KV size explicitly) avoids the problem.

Table 2. Other components.

Component	Deployment	Configuration
LiteLLM gateway	1 pod, NodePort 30400	Master-key auth, OpenAI API, response cache off, 600 s timeout
<code>kv_router</code>	1 pod behind the router Service :8080	Admission: ≤ 64 in flight per decode pod, FIFO queue of 512, 120 s wait limit (429 / 503); prefix hashing in 256-char blocks; prefill hop with <code>max_tokens=1</code>
Mooncake master + store	2 pods (<code>kv-tier</code> namespace)	Master with HTTP metadata server; store pod contributes 32 GiB DRAM with NVMe offload; each vLLM pod contributes a 1 GiB segment
Observability	Prometheus + Grafana	15 s scrape of every vLLM pod, the router and Mooncake
bench-client	1 pod	Runs <code>vllm bench serve</code> and the multi-turn benchmark inside the cluster

Request path (configuration C)

1. The client sends an OpenAI request to the gateway (agents) or directly to the router Service (benchmarks).
2. `kv_router` 's admission queue gives the request a decode slot if a pod has one free (≤ 64 in flight); otherwise it waits in the FIFO queue, or gets HTTP 429 if the queue is full.
3. The router hashes the prompt in blocks and picks the decode pod that has served the longest matching prefix, falling back to the least-loaded pod when load is imbalanced.
4. It sends a copy of the request with `max_tokens=1` to a prefill pod. LMCache stores the prompt's KV in Mooncake.
5. It streams the real request from the chosen decode pod. LMCache loads whatever of the prompt's KV is available from its local CPU cache or Mooncake, and vLLM computes only the rest.

Without the KV tier (C-nokv), step 4's work is discarded: the decode pod recomputes the whole prompt, so the prefill hop only adds latency.

Deployment notes

Issue	Cause	Resolution
vLLM pods crashed at start: <code>libcudart.so.13</code> missing	The <code>-cu129</code> base image paired a CUDA 13 vLLM 0.29 wheel with a CUDA 12.9 PyTorch	All-CUDA-13 image <code>lmcache/vllm-openai:v0.5.5</code> ; the build fails on a CUDA mismatch, and Phase 4 runs a GPU smoke-test pod
Image build failures (PEP 668, <code>libcudart.so.1</code>)	The base venv has no pip; CUDA driver libraries exist only at run time	Install with <code>uv</code> into <code>/opt/venv</code> ; build-time import checks use the CUDA stub library; Mooncake pods run with <code>NVIDIA_VISIBLE_DEVICES=none</code>
All vLLM pods crashed on the first long prompts (Client not available x20, then a segfault)	LMCache 0.5.5 passes Mooncake's dict-based <code>setup()</code> the key <code>master_server_address</code> ; Mooncake 0.3.13 reads <code>master_server_addr</code> and ignores unknown keys, so every vLLM pod dialled the default <code>127.0.0.1:50051</code>	The LMCache config also sets <code>mooncake_master_server_addr</code> (LMCache strips the prefix). Phase 5 fails on any Mooncake client error and pushes a long prompt through the KV tier
Benchmarks reused results from an earlier deployment	Result freshness was tied to only some deploy scripts	Every pool deploy invalidates older results; results with and without the KV tier are labelled C and C-nokv
GPU utilisation panel shows "No data"	DCGM metrics are not reaching Prometheus	Open

Part 2. Capacity on paper

This part works out, from the configuration and the KV sizes vLLM reported, how many requests the cluster holds at once and how long they can be, with and without the KV tier.

2.1 Memory budget per pod

Term	Decode pod (14 GB slice)	Prefill pod (16 GB slice)
Slice × gpu_memory_utilization 0.90	12.6 GiB	14.4 GiB
– FP8 weights	≈ 8.1 GiB	≈ 8.1 GiB
– activations, CUDA graphs, runtime	≈ 0.6 GiB	≈ 0.6 GiB
= KV cache	3.9 GiB = 72,128 tokens	5.7 GiB = 105,840 tokens

Each token of context costs 56 KiB of KV cache (BF16).

2.2 Requests running at once

A request runs only while all its tokens (prompt plus output) sit in a decode pod's KV cache, and vLLM caps each pod at 64 sequences. The binding limit is the smaller of the two: per pod, $\min(64, \text{KV tokens} \div \text{tokens per request})$.

Table 3. Requests that fit at once in the decode pool.

Workload	Tokens / request	Per decode pod	Decode pool, with KV tier (3 × 72k)	Decode pool, C-nokv (72k + 72k + 39k)	Binding limit
Benchmark (512 in + 128 out)	640	64	192	189	max_num_seqs
Agent LLM call (mean)	≈ 1,700	42	127	107	KV cache
Agent call at p95 prompt length	≈ 2,500	28	84	71	KV cache
Full-length request	16,384	4.4	13	11	KV cache

Admission control: the router lets at most 64 requests into each decode pod (192 in total), queues up to 512 more, and rejects beyond that with HTTP 429, so up to 704 requests are accepted at once. For benchmark-sized requests, the router's 192 slots and vLLM's capacity coincide, so overload queues in the router rather than inside vLLM. For long agent prompts the KV cache binds first (≈ 127), so vLLM would start queueing or preempting before the router's limit is reached.

The KV tier does not change these numbers: a running sequence must sit in GPU memory. What it changes is how much finished prompt KV can be kept and reused (2.4).

2.3 Maximum sequence length

The configured maximum is 16,384 tokens (prompt plus output); every pod holds at least 4.4 such sequences. The original 4,096-token limit was too small for agents: the longest agent prompt observed was 9,761 tokens, and typical calls were 1.6k tokens (p95 2.4k).

2.4 Reusable KV with and without the KV tier

Table 4. Where finished prompt KV can live (56 KiB per token).

Tier	Where	Size	KV tokens	Shared across pods
GPU prefix cache (vLLM)	each vLLM pod's GPU memory	the part of 72k–106k tokens not used by running requests	$\leq 72\text{k}$ per decode pod	No
LMCache local CPU	each vLLM pod's RAM	4 GB $\times$ 5 pods	$\approx 75\text{k}$ per pod	No
Mooncake pool (measured)	store pod 32 GiB + 1 GiB per vLLM pod	37 GiB (master_total_capacity_bytes = 39.7 GB)	$\approx 693\text{k}$	Yes
Mooncake NVMe offload	host NVMe	≈ 200 GB configured	≈ 3.5 M	Yes

Table 5. The same stack, without and with the KV tier.

	Without KV tier (C-nokv)	With KV tier (C)
Requests running at once	Set by GPU KV (Table 3)	Same
Longest request	16,384 tokens	Same
Reusable prompt KV	$\leq 72\text{k}$ tokens per decode pod, evicted under load	+ $\approx 75\text{k}$ per pod in CPU RAM, + $\approx 693\text{k}$ shared in Mooncake
Cross-pod reuse	None: a conversation that moves pod is recomputed	Any pod can load KV another pod computed
Prefill $\rightarrow$ decode hop	Adds latency, saves nothing	The decode pod loads the prefill pod's KV
Agent conversations kept warm (2–4k tokens each)	≈ 20 –35 per decode pod	≈ 170 –350 in the Mooncake pool alone

Measured use of the pool. Over the three benchmarks, Mooncake's usage grew from 3.0 GB (204 objects) to 13.7 GB (931 objects), 34% of the pool, with zero failed writes and zero evictions, so the NVMe offload was never needed.

Part 3. Benchmarks: rate4 and rateinf

3.1 Setup

Item	Value
Stacks	C : 2 prefill + 3 decode pods behind kv_router , prefix caching, LMCache/Mooncake KV tier. C-nokv : the same without the KV tier
Client	vllm bench serve (vLLM 0.29) in the bench-client pod, calling the router Service directly (no gateway), streaming /v1/completions
Prompts	Random tokens, 512 in / 128 out, ignore_eos (fixed output length), seed 42; no shared prefixes by design
rate4	100 prompts arriving as a Poisson process at 4 requests/s
rateinf	400 prompts sent at once, so the burst exceeds the router's 192 admission slots
multiturn	20 sessions × 5 turns sharing a ~800-token system prompt and a document; context grows every turn
Order and timing	C-nokv: rate4 and multiturn at 15:28–15:29, rateinf at 16:40. C: rate4 18:30, multiturn 18:31, rateinf 18:34 (local time)
Server metrics	/metrics of every vLLM pod and the Mooncake master snapshotted before and after each run; Grafana at 15 s resolution

3.2 Results

Table 6. rate4: steady load at 4 requests/s.

Metric	Without KV tier	With KV tier	Change
Completed / failed	100 / 0	100 / 0	
Output throughput	464 tok/s	465 tok/s	0%
TTFT p50 / p95 / p99	222 / 273 / 317 ms	254 / 381 / 405 ms	+14% / +39% / +28%
Inter-token latency mean / p95	21.9 / 32.0 ms	21.3 / 28.3 ms	−3% / −12%
End-to-end p50 / p95	3.05 / 3.22 s	3.01 / 3.19 s	−1% / −1%
Prompt tokens served from the KV tier	–	15.5%	

Table 7. rateinf: a burst of 400 requests.

Metric	Without KV tier	With KV tier	Change
Completed / failed	400 / 0	400 / 0	
Duration	21.1 s	19.3 s	−9%
Request throughput	18.9 req/s	20.7 req/s	+9%
Output throughput	2,424 tok/s	2,646 tok/s	+9%
TTFT p50 / p95 / p99	9.50 / 15.63 / 18.21 s	8.69 / 16.78 / 16.80 s	−9% / +7% / −8%

Metric	Without KV tier	With KV tier	Change
Inter-token latency mean / p95	51.9 / 197.1 ms	30.6 / 40.1 ms	−41% / −80%
End-to-end p50 / p95	17.7 / 19.3 s	14.3 / 19.3 s	−19% / 0%
GPU prefix-cache hit rate	0.1%	23.8%	
Prompt tokens served from the KV tier	–	32.8%	
Preemptions	2	0	
Router queue peak (Grafana, without KV tier)	≈ 210	–	

Table 8. multiturn: 100 requests (20 sessions × 5 turns).

Metric	Without KV tier	With KV tier	Change
Output throughput	631 tok/s	622 tok/s	−1%
TTFT p50 / p95	208 / 311 ms	285 / 492 ms	+37% / +58%
Inter-token latency p95	27.1 ms	27.4 ms	+1%
End-to-end p95	3.17 s	3.28 s	+3%
GPU prefix-cache hit rate	85.6%	85.2%	
Remaining prompt tokens served from the KV tier	–	15.8%	

Table 9. Activity in the KV tier (C), from the before/after metrics snapshots.

Run	Mooncake writes (batches started / completed)	Mooncake reads (batches)	Objects in Mooncake after the run	Pool used after the run
rate4	169 / 100	31	204	3.0 GB
multiturn	89 / 80	22	326	4.8 GB
rateinf	456 / 344	204	931	13.7 GB

3.3 Steady load (rate4)

At 4 requests/s the cluster is far from its limits: about 11 requests are in flight ($3.6 \text{ req/s} \times 3.05 \text{ s}$, Little's law) against 192 slots. Without the KV tier, every request got its first token in about a quarter of a second (p99 317 ms) and streamed at $\approx 46 \text{ tokens/s}$, so a 128-token answer took just over 3 s. Nothing queued and nothing was preempted.

With the KV tier, throughput and streaming speed are unchanged, but **time to first token rises** (p50 +14%, p95 +39%). Each request's prefill hop now stores its KV in Mooncake (169 write batches for 100 requests), and each decode request first looks up and loads what is available. 15.5% of prompt tokens were loaded from the tier instead of recomputed, well short of the whole prompt: LMCache works in 256-token chunks and vLLM must recompute at least the final token, so at most one of a 512-token prompt's two chunks is reusable, and a lookup can run before the prefill pod's asynchronous store has finished. At this load the GPU has spare compute, so recomputing is cheaper than the round trip to the tier.

3.4 A 400-request burst (rateinf)

The burst shows the admission design working as sized in Part 2. About 190 requests were admitted at once (running requests reached ≈ 60 per decode pod), and the router queue peaked at ≈ 210 , which is 400 minus the admitted requests (Figure 9). vLLM's own waiting count stayed near zero (Figure 6). Every request completed.

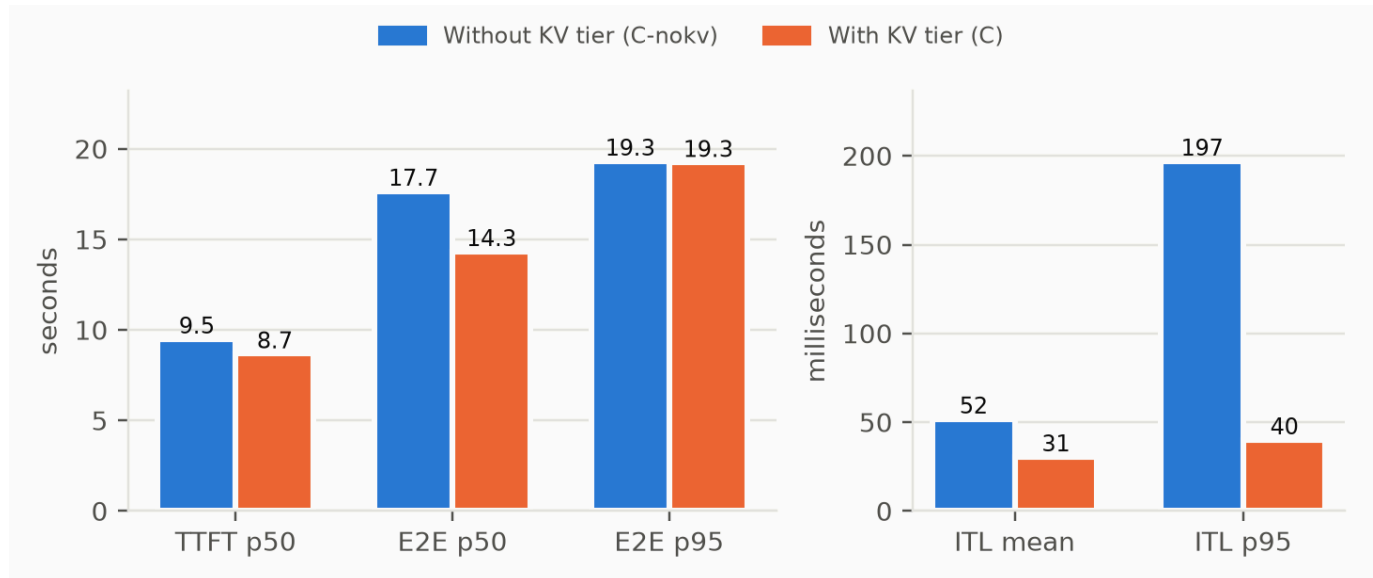


Figure 3. The 400-request burst without (blue) and with (orange) the KV tier.

With the KV tier, the burst ran faster:

- **Throughput +9%** (2,424 $\rightarrow$ 2,646 output tokens/s), and the burst finished in 19.3 s instead of 21.1 s.
- **Inter-token latency fell sharply:** mean -41% , p95 -80% (197 $\rightarrow$ 40 ms). Without the tier, the p95 was dominated by the pod with the smallest KV cache (Figure 5).
- **Median end-to-end latency -19%** (17.7 $\rightarrow$ 14.3 s); p95 unchanged, because the last requests still waited for a free slot.
- **Preemptions 2 $\rightarrow$ 0.**
- **TTFT** changed little (p50 -9% , p95 $+7\%$). It is dominated by time in the router queue: about 210 requests wait for the first wave to finish.

Attribution. Two other differences between the runs favour the KV-tier run, so its gain is an upper bound on what the tier itself contributes:

1. **More even GPU memory.** Without the tier, `vllm-decode-2` had 39,440 tokens of KV against 72,128 on its peers; it was the pod whose KV usage reached $\approx 80\%$, that was preempted, and whose inter-token latency spiked (Figures 5–6). With the tier, all three decode pods had 72,128 tokens.
2. **Prompt reuse between runs.** The benchmark's prompts come from a fixed seed, so the burst repeats the earlier rate4 prompts. With the tier, the burst ran 4 minutes after rate4 and found 23.8% of prompt tokens in the GPU prefix cache and 32.8% in the KV tier; without the tier it ran an hour after rate4, after other traffic had cleared the cache, and found 0.1%.

A clean attribution needs both variants deployed the same way (pods started one at a time) and the burst run with a fresh seed.

3.5 Multi-turn conversations

Multi-turn traffic shows what prefix caching buys: 85% of prompt tokens were already in the GPU prefix cache, in both variants, because `kv_router` keeps each session on the pod that holds its prefix. Without the KV tier, the first turn averaged 278 ms to first token and later turns 191–215 ms, although their prompts were 11–46% longer.

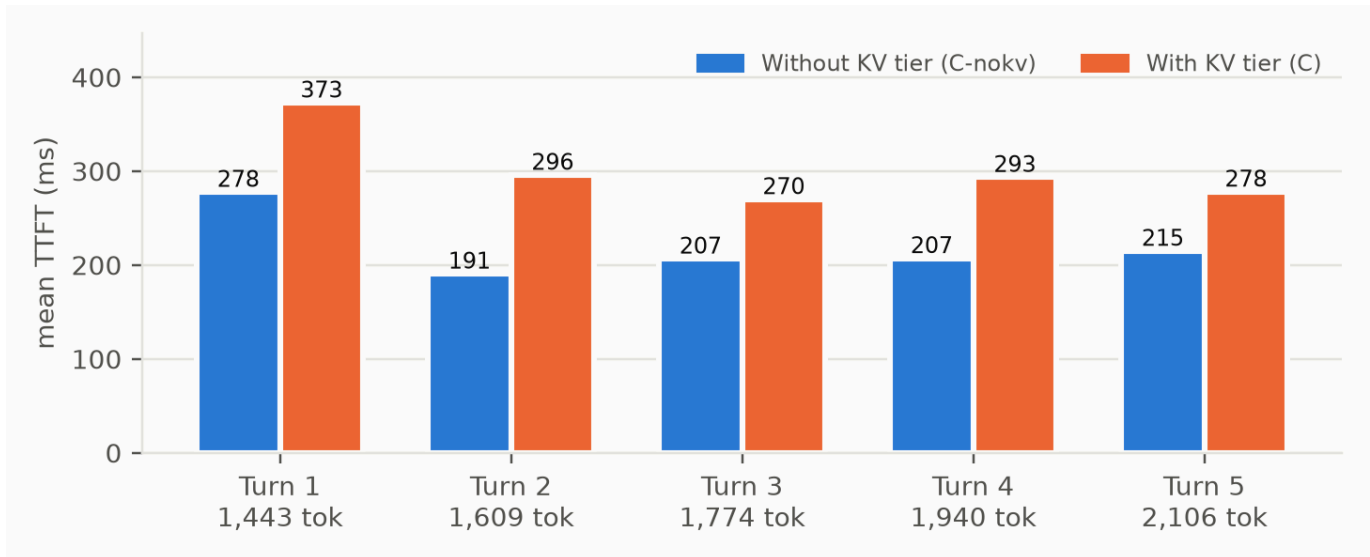


Figure 4. Mean time to first token by turn, without (blue) and with (orange) the KV tier; labels show the mean prompt length per turn.

With the KV tier, every turn is slower (+29% to +55%). The GPU prefix cache already serves the reuse, so the tier has little left to add (15.8% of the remaining tokens), while every request pays for storing its KV and looking it up. The tier's value for conversations appears where the GPU cache cannot help: sessions that move between pods, more sessions than fit in GPU memory, or sessions resumed after their GPU cache was evicted. These benchmarks don't produce any of those situations.

3.6 Grafana views

The dashboard captures cover 16:22–16:52, **on the stack without the KV tier**. Three bursts are visible: the agent evaluation at concurrency 4 ($\approx$ 16:28–16:30), at concurrency 8 ($\approx$ 16:34–16:35), and the 400-request rateinf burst ($\approx$ 16:40–16:41).

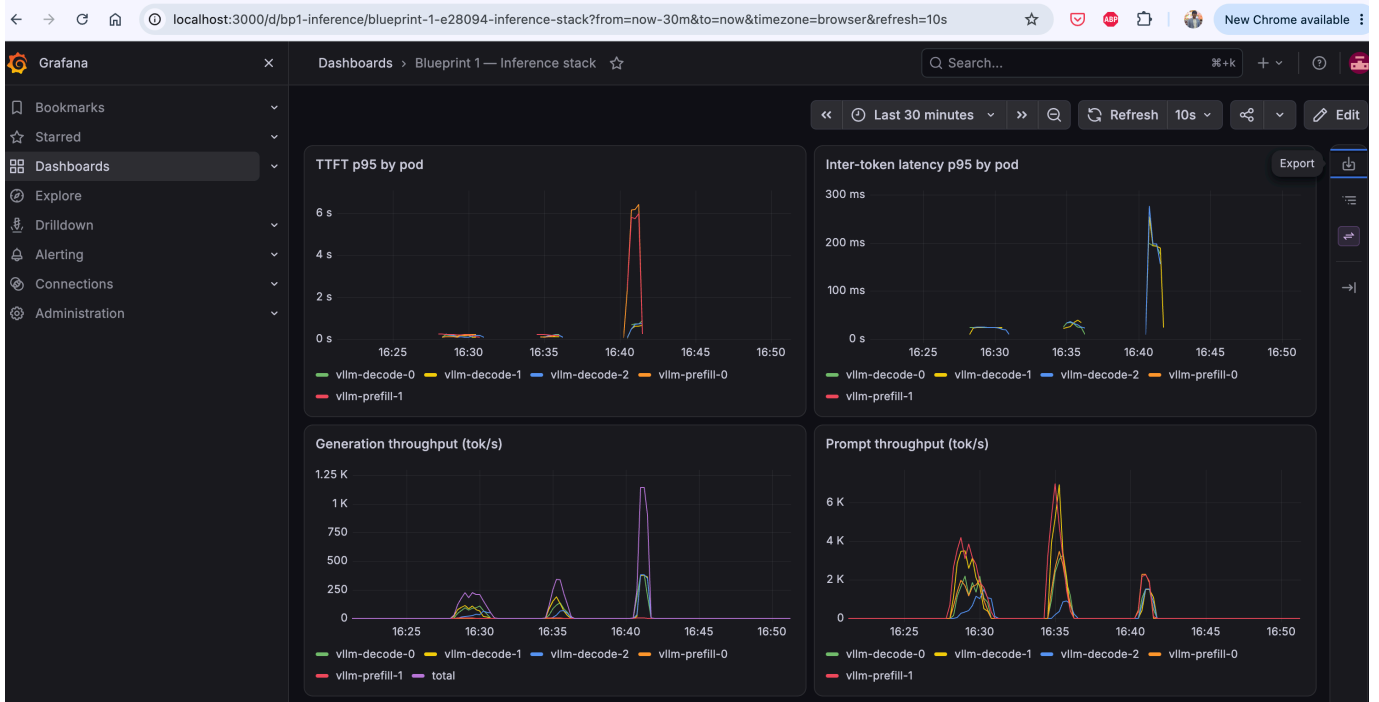


Figure 5. Latency and throughput per pod. The burst drives TTFT p95 to ≈ 6 s and inter-token p95 to ≈ 280 ms on `vllm-decode-2`; the agent runs stay sub-second. The throughput panels use a 1-minute rate, which spreads out the 21-second burst, so their ≈ 1.1 k tokens/s peak under-reads the measured 2,424 tokens/s.

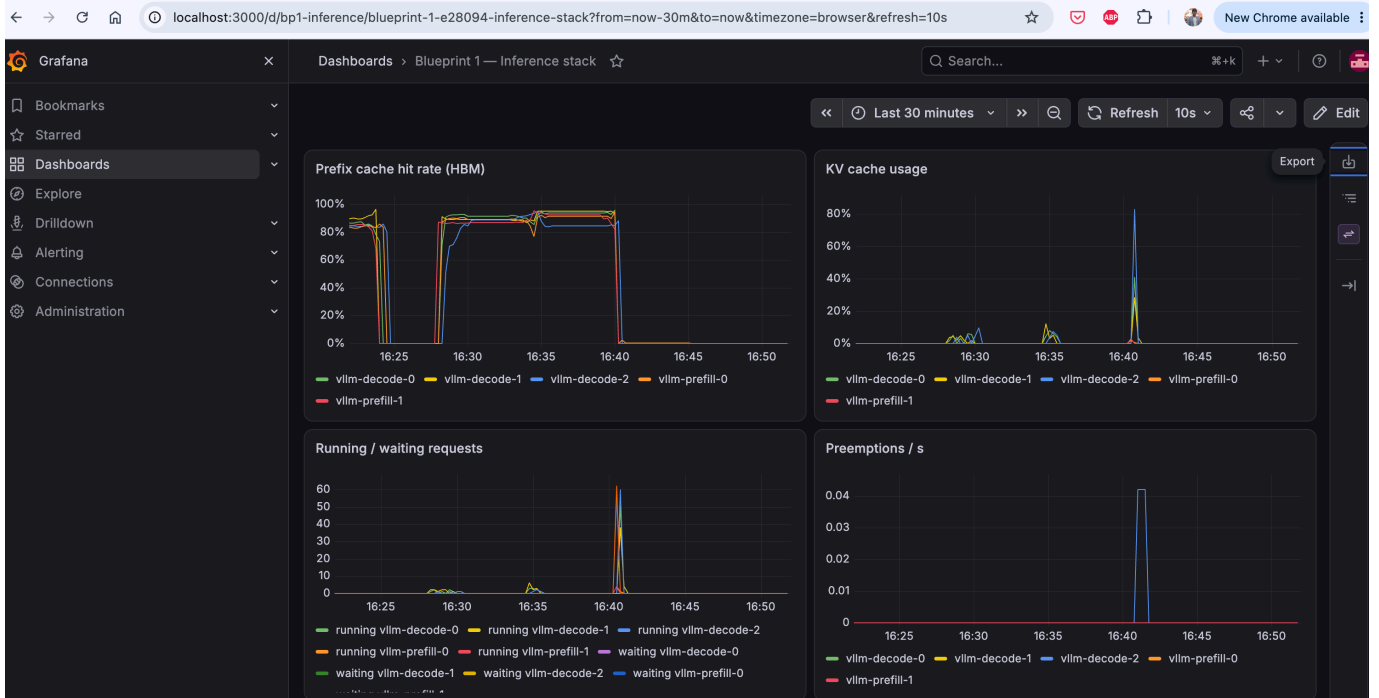


Figure 6. Prefix-cache hit rate holds at 85–95% through the agent runs and falls to ≈ 0 for the random-prompt burst. At the burst, running requests reach ≈ 60 per pod while vLLM's waiting count stays near 0; KV usage and preemptions peak on `vllm-decode-2`, the pod with the smallest KV cache.

Part 4. Validation with agent runs

4.1 Setup

Item	Value
Path	Laptop (CrewAI) → SSH tunnel → LiteLLM gateway → kv_router → prefill/decode pools, without the KV tier
Workload	All 12 graded questions per run; each question in its own process with its own crew; the three agents run in sequence and wait for each reply
Concurrency	4 questions at a time (runs 1–3) and 8 (run 4)
Model settings	Run 1: temperature 0. Runs 2–4: temperature 0.3, top_p 0.8, top_k 20, repetition penalty 1.05. Runs 3–4 also stop each reply at <code></tool_call></code>
Measurement	CrewAI event hooks record every LLM call (latency, tokens, output); tools count their own calls; answers are graded against the database; Prometheus is queried over each run's window
Outputs	<code>agent_runs.csv</code> (one row per run), <code>agent_questions.csv</code> (one row per question), per-call traces

4.2 Results across runs

Each run changed one thing, based on what the previous run's traces showed.

Table 10. Agent evaluation runs (12 questions each).

	Run 1	Run 2	Run 3	Run 4
Change	Baseline, temperature 0	+ sampling, guardrails	+ one tool call per reply, schema tools for all agents	Run 3 at concurrency 8
Correct	1 / 12	3 / 12	6 / 12	5 / 12
LLM calls per question	8.9	17.8	32.3	30.3
Tool calls per question	17.1	30.5	29.0	26.8
Guardrail retries (total)	0	9	8	9
Wall time per question p50 / p95	37.9 / 49.2 s	62.4 / 113.2 s	32.9 / 72.9 s	32.4 / 58.6 s
Questions per minute	6.1	3.3	5.4	10.5
Prompt tokens / s offered	1,779	1,986	4,806	8,312
GPU prefix-cache hit rate	77.8%	84.8%	88.6%	93.1%
vLLM TTFT (mean)	82 ms	84 ms	75 ms	80 ms
Router queue peak / vLLM waiting / preemptions	0 / 0 / 0	0 / 0 / 0	0 / 0 / 0	0 / 0 / 0

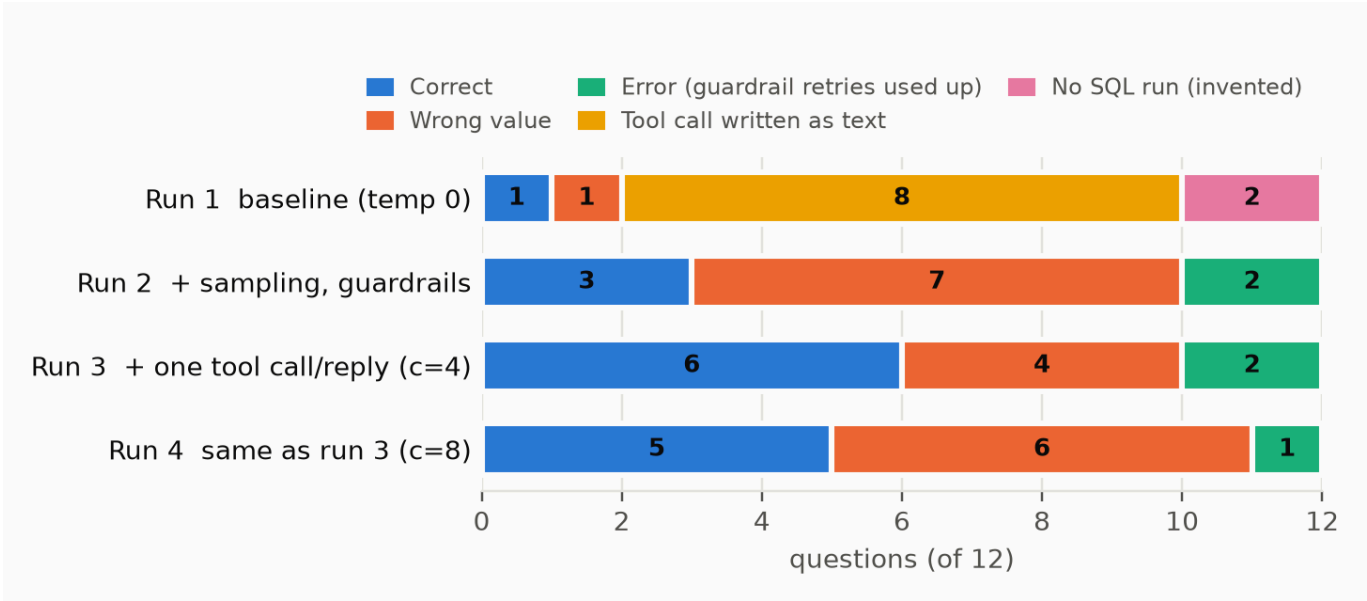


Figure 7. Outcome of each question per run, by failure reason. Mechanical failures (tool calls written as text, answers with no SQL) disappear by run 2; runs 3–4 fail mostly on wrong values, plus runs that used up their guardrail retries.

4.3 Why answers were wrong, and what fixed it

Every LLM call was traced, so each failure could be attributed.

Finding (run)	Evidence	Fix
Tool calls written as text (run 1: 8 of 11 misses)	The final "answer" was a JSON tool call or <code><tool_call></code> tags; some outputs degenerated into repeated <code>< im_start ></code> tokens under greedy decoding	Qwen-style sampling instead of temperature 0; stop at chat-template tokens; guardrails reject a text tool call and make the agent retry
Answers invented without SQL (run 1: 2)	q12 reported a non-existent column and made-up counts with zero queries run	Guardrails require the Analyst and the Reviewer to run queries
Batched tool calls rejected all-or-nothing (run 2)	All 14 text tool-call replies batched 5–14 calls; 12 had a malformed or cut-off call, and vLLM's hermes parser then returns the whole batch as text; 11 replies hit the 1,024-token limit	Stop generation at <code></tool_call></code> : one tool call per reply (vLLM parses the unclosed call)
Schema guessing (run 2)	The Planner's exploration was lost to the batching failure; agents guessed tables (<code>tickets</code> , <code>order_status</code>) and answered <code>NULL</code> , <code>0.0</code> or "hypothetical" values	Planner guardrail (must describe tables), schema tools for every agent, SQL errors list the real tables, Reviewer rejects <code>NULL</code> /hypothetical answers
Over-interpretation (run 2, q01)	"Customers in the West region" answered as customers with orders (102 vs 103)	Instruction to answer the question as literally worded

Table 11. Per-question outcome.

Question	Difficulty	Run 1	Run 2	Run 3	Run 4
q01	easy	OK	wrong value	OK	OK

Question	Difficulty	Run 1	Run 2	Run 3	Run 4
q02	medium	text tool call	wrong value	OK	OK
q03	medium	text tool call	OK	OK	wrong value
q04	medium	wrong value	error	wrong value	wrong value
q05	medium	text tool call	OK	wrong value	wrong value
q06	hard	text tool call	wrong value	error	OK
q07	hard	text tool call	OK	OK	wrong value
q08	hard	text tool call	wrong value	error	wrong value
q09	hard	text tool call	wrong value	wrong value	error
q10	hard	text tool call	wrong value	wrong value	wrong value
q11	hard	no SQL	error	OK	OK
q12	hard	no SQL	wrong value	OK	OK

After the fixes, four questions are reliably correct (q01, q02, q11, q12) and two are never right (q04, q10); the other six flip between runs. With a single pass over 12 questions at temperature 0.3, a difference of one or two answers is normal variation, so run 3's 6/12 and run 4's 5/12 are the same result. Firmer comparisons need repeated runs (`--repeat 3`).

4.4 Cluster behaviour under agent load

- **Agent traffic is prefill-heavy and highly cacheable.** Each call sent $\approx 1.6k$ prompt tokens and received ≈ 64 (26:1), and consecutive calls re-send the same growing conversation. The GPU prefix-cache hit rate reached 89–93%, and `kv_router`'s prefix-match ratio held at $\approx 90\%$ (Figure 8): conversations stayed on the pod that already had their prefix.
- **Latency stayed flat.** vLLM's mean TTFT was 75–84 ms in every run; inter-token p95 stayed around 25–40 ms (Figure 5).
- **Doubling concurrency doubled throughput.** From 4 to 8 concurrent questions, throughput rose from 5.4 to 10.5 questions/min and offered prompt load from 4.8k to 8.3k tokens/s, while the median time per question stayed at ≈ 32 s. The cluster was far from saturation.
- **The admission queue stayed at 0.** Each crew waits for every reply before its next call, so 8 concurrent questions put at most 8 requests in flight against 192 slots (≈ 3 per pod; running requests ≤ 5 per pod in Figure 6). A question's ≈ 33 s is mostly its ≈ 30 sequential LLM round trips, not waiting for the GPU.
- **One tool call per reply trades more calls for reliability.** LLM calls per question roughly doubled (17.8 $\rightarrow$ 32.3), yet wall time fell (62 $\rightarrow$ 33 s p50) because each reply is short and no longer fails. For the cluster this means more, smaller, highly cacheable requests.

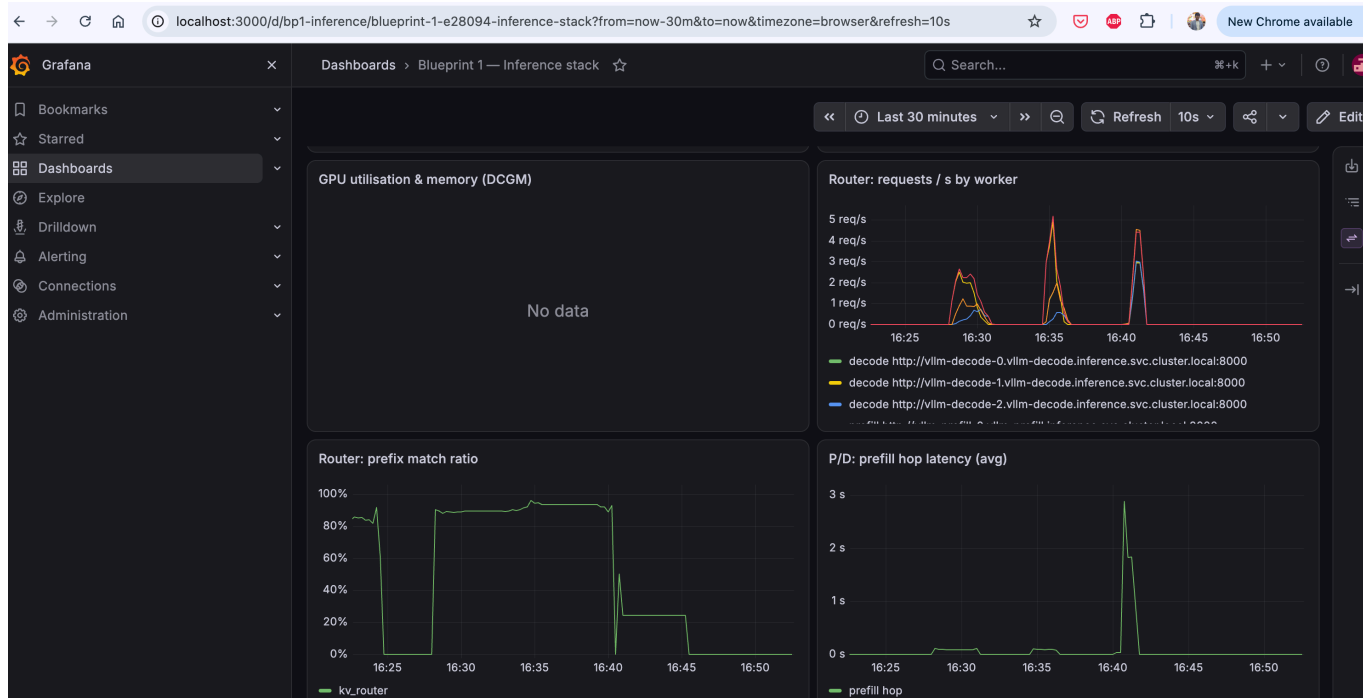


Figure 8. Router views (stack without the KV tier). Requests spread across the three decode pods; kv_router's prefix-match ratio holds at $\approx 90\%$ for agent traffic and drops for the random-prompt burst. The GPU utilisation panel is empty because DCGM metrics were not collected.

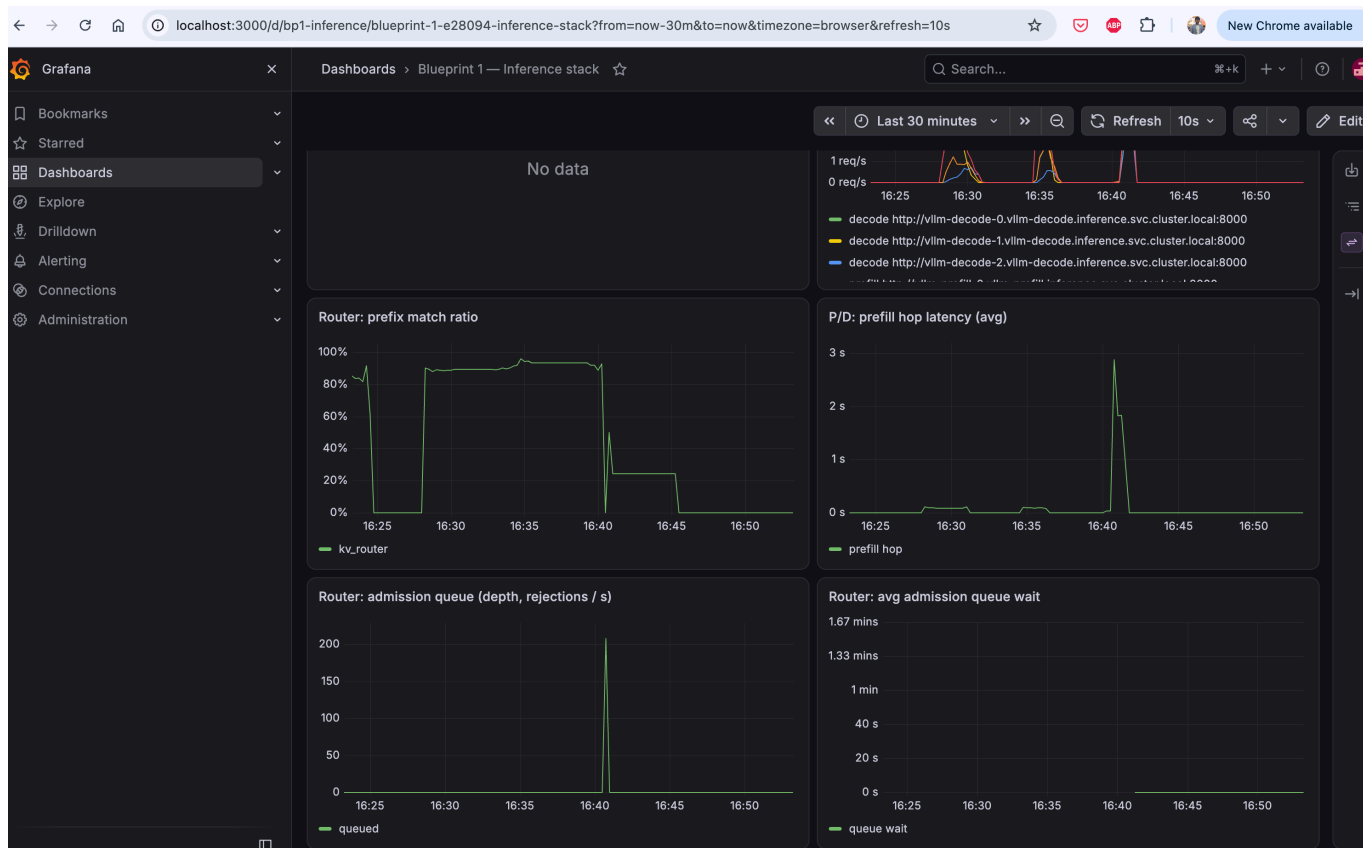


Figure 9. Admission queue (stack without the KV tier). Only the 400-request burst at 16:40 queues (≈ 210); the agent runs never do. The queue-wait panel misses the burst because the router's wait counters are created on first use, so Prometheus's first

sample already includes it.

4.5 Conclusions and next steps

- **The stack handles agent workloads at this scale comfortably.** Agent traffic caches extremely well, stays at sub-100 ms TTFT, and at concurrency 8 uses a small fraction of capacity.
- **Bursts are handled by design.** The admission queue holds overload in the router instead of vLLM; the price is queueing latency for requests beyond the first wave.
- **The KV tier helps under load and costs at light load.** It raised burst throughput and cut decode latency, but at light load it added 14–37% to median TTFT and 39–58% to p95, where the GPU prefix cache already serves the reuse. It pays off when prompts are reused across pods, sessions outgrow GPU memory, or load is high.
- **Run the agent evaluation with the KV tier.** Agent traffic ($\approx$ 1.6k-token prompts, 26:1 prompt-to-output ratio, heavy reuse) is the workload the tier is designed for, and the runs above predate it.
- **Make the comparison clean.** Deploy both variants the same way (pods started one at a time, so KV allocation is even), use a fresh seed for the burst, and repeat each run.
- **Complete the ablation.** Run configurations A (one pod, no router) and B (decode pods with `sglang_router`) with the same benchmarks and agent evaluation, and fix DCGM collection so GPU utilisation can be reported.

Appendix. Reproducing the runs

On the node, from `blueprint1/`:

```
./scripts/05_deploy_vllm.sh C                                # pools with the KV tier
    (ENABLE_KV_TIER_C="true")
PATTERNS="rate4 multiturn" ./scripts/08_run_benchmarks.sh quick
QUICK_NUM_PROMPTS=400 PATTERNS=rateinf ./scripts/08_run_benchmarks.sh quick
# Without the KV tier: ENABLE_KV_TIER_C="false" in config.env, redeploy, rerun (results are
    labelled C-nokv)
```

On the laptop, with `./scripts/laptop.sh tunnel` open:

```
./scripts/laptop.sh fetch                                    # -> bench-results/quick, bench-
    results/full
cd ../analyst_crew
.venv/bin/python run.py eval --concurrency 4 --prometheus http://localhost:9090 --label "C kv-
    tier"
.venv/bin/python run.py aggregate                            # rebuild agent_runs.csv /
    agent_questions.csv
```

Artefact	Location
Benchmark results and summary	<code>bench-results/quick/</code> (<code>results_<config>_<pattern>.json</code> , metrics snapshots, <code>summary.md</code> , <code>summary.csv</code>)
Agent runs	<code>bench-results/agent/<run_id>/</code> (<code>meta.json</code> , <code>results.jsonl</code> , per-question traces)
Report tables	<code>agent_runs.csv</code> , <code>agent_questions.csv</code>
Configuration	<code>blueprint1/config.env</code>